Evolutionary Computation: Foundations

The field of evolutionary computation draws its core principles from biological evolution, encoding candidate solutions as individuals within a population and iteratively refining them through selection, recombination, and mutation. What began as independent lines of research in the United States and Germany during the 1960s and 1970s has since matured into a unified discipline with dedicated conferences, journals, and a rich theoretical literature. This section traces the intellectual origins of evolutionary algorithms, surveys the principal paradigms, and examines the theoretical tools that have shaped the field.

Origins: Evolution Strategies in Germany

The earliest experiments with evolution-inspired optimization took place at the Technical University of Berlin in the mid-1960s. Ingo Rechenberg, working alongside Hans-Paul Schwefel and Peter Bienert, sought optimal shapes for bodies in a wind tunnel and arrived at the idea of applying random perturbations to design parameters in analogy with biological mutation [Rechenberg1973]. Rechenberg formalized this approach in his 1973 monograph *Evolutionsstrategie*, which introduced the $(1 + 1)$-ES, a minimal scheme in which a single parent produces a single offspring through Gaussian mutation, and the offspring replaces the parent only if it achieves superior fitness. The crucial insight was the *one-fifth success rule*, an adaptive heuristic for adjusting mutation step sizes based on the observed fraction of successful mutations.

Schwefel extended the framework substantially, introducing population-based variants with recombination operators and, most importantly, the concept of *self-adaptation*, in which strategy parameters such as mutation step sizes are themselves subject to evolution [Schwefel1981]. His 1981 English-language text *Numerical Optimization of Computer Models* brought evolution strategies to an international audience and established the (μ, λ) and $(\mu + \lambda)$ selection notations that remain standard today. A later survey by Beyer and Schwefel provided a comprehensive theoretical treatment of the convergence properties and design principles underlying these methods [Beyer2002].

Genetic Algorithms and the Schema Theorem

Independently, John Holland at the University of Michigan developed a different evolutionary framework motivated not primarily by engineering optimization but by the study of adaptation itself. His 1975 book *Adaptation in Natural and Artificial Systems* introduced the *genetic algorithm* (GA), in which candidate solutions are encoded as fixed-length binary strings and transformed through selection, crossover, and mutation operators modeled on the mechanisms of population genetics [Holland1975]. Holland's formulation emphasized the role of *schemata*, templates defined over the binary alphabet that represent subsets of the search space sharing common features at certain positions. The *Schema Theorem* provided an inequality bounding the expected number of instances of a schema in the next generation, showing that short, low-order schemata with above-average fitness tend to receive exponentially increasing representation over successive generations.

The Schema Theorem was popularized and extended by David Goldberg, Holland's student, whose 1989 textbook *Genetic Algorithms in Search, Optimization and Machine Learning* became the field's most widely read introduction [Goldberg1989]. Goldberg articulated the *Building Block Hypothesis*, which posited that genetic algorithms achieve their power by assembling short, high-fitness schemata into progressively better solutions through crossover. Kenneth De Jong's 1975 dissertation further cemented the role of genetic algorithms in function optimization by establishing a standard suite of test functions and systematically studying the

effects of population size, crossover rate, and mutation probability on algorithm performance [DeJong1975].

However, the Schema Theorem and the Building Block Hypothesis have attracted significant criticism over the decades. The theorem is an inequality rather than an equality, and it holds exactly only in the limit of infinite populations, making it a weak predictor of finite-population behavior. Critics have observed that the theorem cannot distinguish between problem instances where genetic algorithms perform well and those where they fail, limiting its explanatory power [Fogel1995]. The Building Block Hypothesis, while intuitive, has been challenged by empirical studies demonstrating that schema fitness is highly context-dependent and that the static assembly of building blocks does not reliably account for GA dynamics on deceptive or multimodal landscapes.

Evolutionary Programming

A third independent lineage originated with Lawrence Fogel in San Diego in the early 1960s. Fogel's evolutionary programming (EP) approach evolved finite-state automata to perform prediction tasks, treating the behavioral repertoire of a program rather than its syntactic structure as the object of selection [Fogel1966]. Unlike genetic algorithms, evolutionary programming did not employ crossover, relying instead on mutation as the sole variation operator and tournament selection as the primary survival mechanism. David Fogel later extended this line of work and placed it within the broader context of evolutionary computation in his 1995 monograph *Evolutionary Computation: Toward a New Philosophy of Machine Intelligence* [Fogel1995].

Genetic Programming

John Koza's 1992 book *Genetic Programming: On the Programming of Computers by Means of Natural Selection* introduced a paradigm in which the individuals subject to evolution are themselves computer programs, typically represented as parse trees [Koza1992]. Koza demonstrated that genetic programming (GP) could automatically discover programs solving a diverse range of problems, from symbolic regression and classification to control system design, without requiring the user to specify the form of the solution in advance. The key operators in GP are subtree crossover, which exchanges randomly selected subtrees between two parent programs, and subtree mutation, which replaces a randomly selected subtree with a newly generated random subtree. Koza's work represented the first large-scale, non-trivial demonstration of automatic programming since the concept was discussed in the 1940s, and it opened a productive research program that continues to generate novel applications in program synthesis and symbolic discovery.

Unifying the Field

By the mid-1990s, the parallel development of evolution strategies, genetic algorithms, evolutionary programming, and genetic programming had created a fragmented landscape with overlapping terminology and partially redundant theory. Thomas Back's 1996 monograph *Evolutionary Algorithms in Theory and Practice* was among the first to present all three major paradigms within a unified formal framework, comparing their selection mechanisms, representation schemes, and variation operators and identifying the common algorithmic skeleton that underlies them all [Back1996]. Eiben and Smith's 2003 textbook *Introduction to Evolutionary Computing* further consolidated this unified perspective for a pedagogical audience, organizing the material around the shared concepts of representation, fitness evaluation, parent selection, recombination, mutation, and survivor selection [Eiben2003].

Institutional developments mirrored this intellectual convergence. The *IEEE Transactions on Evolutionary Computation*, founded in 1997 with David B. Fogel as its inaugural editor-in-chief, provided the field's first dedicated archival journal. Two years later, the *Genetic and Evolutionary Computation Conference* (GECCO) was established in 1999 through the merger of the International Conference on Genetic Algorithms (ICGA, held biennially since 1985) and the Annual Genetic Programming Conference (GP, held since 1996). GECCO was subsequently adopted by the ACM Special Interest Group on Genetic and Evolutionary Computation (SIGEVO) in 2005, completing the organizational unification of a discipline that had spent three decades developing along separate paths.

Core Mechanisms

Across all paradigms, evolutionary algorithms share a common procedural structure. A *population* of candidate solutions is initialized, typically at random, and then iteratively transformed through three operations. *Selection* determines which individuals contribute genetic material to the next generation, with mechanisms ranging from fitness-proportionate (roulette wheel) selection in classical GAs to deterministic truncation selection in evolution strategies and tournament selection in evolutionary programming. *Recombination* (or crossover) combines information from two or more parents to produce offspring, exploiting the hypothesis that fit parents share complementary partial solutions. *Mutation* introduces stochastic perturbations, maintaining diversity in the population and enabling the exploration of regions not reachable through recombination alone. The balance between *exploitation* of currently known good solutions and *exploration* of unvisited regions of the search space is the central tension governing the design of any evolutionary algorithm, and decades of theoretical and empirical work have been devoted to understanding how operator rates, population sizes, and selection pressures mediate this tradeoff.

The foundations surveyed here provide the algorithmic vocabulary and theoretical grounding upon which modern work in evolutionary discovery of reinforcement learning algorithms builds. The capacity of evolutionary methods to search large, structured, combinatorial spaces without gradient information makes them natural candidates for the meta-optimization of algorithmic components, a theme that subsequent sections of this review will develop in detail.

References

[Holland1975] J.H. Holland. *Adaptation in Natural and Artificial Systems.* University of Michigan Press, Ann Arbor, 1975. (2nd edition, MIT Press, 1992.)

[Rechenberg1973] I. Rechenberg. *Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution.* Frommann-Holzboog, Stuttgart, 1973.

[Schwefel1981] H.-P. Schwefel. *Numerical Optimization of Computer Models.* Wiley, Chichester, 1981. (Original German text, 1977; English translation by M. Finnis.)

[Beyer2002] H.-G. Beyer and H.-P. Schwefel. "Evolution strategies: A comprehensive introduction." *Natural Computing*, 1(1):3–52, 2002. DOI: 10.1023/A:1015059928466.

[Goldberg1989] D.E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning.* Addison-Wesley, Reading, MA, 1989.

[DeJong1975] K.A. De Jong. "An analysis of the behavior of a class of genetic adaptive systems." Doctoral dissertation, University of Michigan, Ann Arbor, 1975.

[**Fogel1966**] L.J. Fogel, A.J. Owens, and M.J. Walsh. *Artificial Intelligence through Simulated Evolution*. John Wiley, New York, 1966.

[**Fogel1995**] D.B. Fogel. *Evolutionary Computation: Toward a New Philosophy of Machine Intelligence*. IEEE Press, Piscataway, NJ, 1995.

[**Koza1992**] J.R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, 1992.

[**Back1996**] T. Back. *Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms*. Oxford University Press, New York, 1996.

[**Eiben2003**] A.E. Eiben and J.E. Smith. *Introduction to Evolutionary Computing*. Springer-Verlag, Berlin, 2003.